

Construction des différentes heuristiques

Approche par compilation de connaissances

Auteur : Mathis Francine-Habas

Structure : LAAS-CNRS

Date : 7 août 2026

Version : 3.0

Table des matières

1. Préambule

Préalablement aux algorithmes d'extraction présentés dans ce document, le Sous-Problème est résolu via un algorithme de programmation dynamique forward. Nous définissons la matrice des potentiels π , où $\pi_{t,j}$ représente le coût maximal cumulé depuis l'origine ($t = 0$) jusqu'à la fin de la période t , en ayant consommé exactement j unités de budget d'incertitude. Le coût de transition d'un état à un autre est donné par la matrice c . Dans la suite du rapport, nous utilisons une variable nommée τ . Il s'agit d'un paramètre de relaxation par rapport au coût optimal. Il permet notamment de définir l'ensemble relaxé contenant les chemins optimaux et sous-optimaux. Cette variable peut être utilisée au besoin, mais n'est utilisée que dans les heuristiques KCRDK/L et KCHOG/L.

2. KCU (KC avec chemin optimal unique)

2.1. Introduction

L'heuristique KCU a été conçue pour allier la puissance de la factorisation avec la maîtrise combinatoire de la méthode de Benders Adverse (BA). Contrairement à l'approche BAE qui inonde le Maître d'un sous-graphe dense représentant de multiples scénarios, KCU limite strictement l'extraction à **un unique scénario optimal** par itération. KCU utilise le même problème Maître que les méthodes KC décrites dans ce document ; seul l'algorithme d'extraction du Sous-Problème adverse diffère.

Note : KCU ne regarde que les chemins strictement optimaux donc nous ne gérons aucun ensemble relaxé. Par conséquent, le coefficient d'approximation τ n'a aucune influence sur cette méthode.

2.2. Extraction et Factorisation

Bien que KCU ne renvoie qu'un seul scénario, son avantage majeur réside dans son intégration au Maître. Chaque nouveau scénario optimal extrait est injecté dans le graphe partiel de budget global via une opération de fusion. Au fil des itérations, les tronçons temporels communs entre les différents scénarios optimaux successifs se superposent et sont factorisés en mémoire. Cela permet au Maître de maîtriser sa complexité, tout en résolvant un Graphe Orienté Acyclique (DAG) sans redondance.

Pour l'extraction de ce chemin unique, l'utilisation d'un algorithme de recherche en profondeur (DFS) est superflue. Nous implémentons une approche par remontée gloutonne, détaillée dans l'Algorithme ??.

Note : cette approche gloutonne est logiquement un cas particulier du DFS ($N = 1, L = 1$) où N est le nombre de chemins à remonter au Maître et L la limite d'exploration du DFS. Cependant, pour gagner du temps de calcul, nous avons préféré opter pour une approche non récursive. Mais si nous souhaitons tester tous les modèles sur la même base, la modification est triviale.

- **Identification de la racine (l. 3 à 8) :** L'algorithme parcourt les nœuds terminaux ($t = T$) et sélectionne le premier budget i vérifiant $|\pi_{T,i} - ub_cost| < \epsilon$, où ϵ est une tolérance numérique fixée à 10^{-4} . La recherche s'arrête dès ce nœud trouvé.
- **Remontée gloutonne (l. 11 et suivantes) :** À chaque pas de temps suivant, l'algorithme évalue les antécédents possibles. Dès qu'un arc respectant la condition d'optimalité est identifié, il est validé et ajouté au sous-graphe.
- **Déterminisme :** L'ordre séquentiel d'évaluation (budgets croissants, puis type 0, puis type 1)¹ rend cette extraction strictement déterministe. En cas de dégénérescence, c'est-à-dire lorsque plusieurs antécédents $(t - 1, i)$ conduisent au même potentiel optimal $\pi_{t,i}$,

1. Le problème a été modélisé en séparant les deux types de coût. Nous avons le type 0 qui modélise le choix d'attaquer en surstockage et le type 1 la rupture.

l'algorithme sélectionne systématiquement le premier rencontré selon cet ordre lexicographique. Ce mécanisme assure la reproductibilité de l'extraction. Cependant, il est important de noter que KCU ne garantit pas l'exhaustivité des chemins optimaux. Elle extrait un unique chemin optimal parmi l'ensemble des solutions de coût minimal. Cette limitation est compensée par sa faible complexité (voir Section ??).

2.3. Algorithme glouton

Les Algorithmes ?? et ?? ont une complexité en $\mathcal{O}(T\Gamma)$. La remontée s'effectue sur T périodes. À chaque période, nous explorons au pire l'ensemble des budgets antérieurs possibles, limité par Γ . Le temps de vérification des transitions est en $\mathcal{O}(1)$.

Algorithm 1: Greedy extraction of the optimal scenario (KCU)

Require: Dual matrix π , Costs matrix c , Optimal cost ub_cost , Tolerance ϵ
Ensure: Partial graph $\mathcal{G}_{sub}$ containing the optimal scenario

```

1  $\mathcal{G}_{sub} \leftarrow$  Initialize an empty graph
2  $j_{curr} \leftarrow -1$ 
3 for  $i = 0$  to  $\Gamma$ 
4   if  $|\pi[T][i] - ub\_cost| < \epsilon$ 
5      $j_{curr} \leftarrow i$  // Search for the first optimal terminal node
6   break
7 if  $j_{curr} \neq -1$ 
8   Add the arc  $(T, j_{curr}) \rightarrow S_{end}$  in  $\mathcal{G}_{sub}$ 
9   for  $t = T$  downto 1
10     $arc\_found \leftarrow \text{false}$ 
11    for  $i = 0$  to  $j_{curr}$  // Search for optimal antecedents
12
13      if The transition  $(i \rightarrow j_{curr})$  is physically valid
14        if  $|\pi[t][j_{curr}] - (\pi[t-1][i] + c[t][i][j_{curr}][0])| < \epsilon$ 
15          Add the overstocking arc in  $\mathcal{G}_{sub}$  // Type 0
16           $j_{curr} \leftarrow i$ 
17           $arc\_found \leftarrow \text{True}$ 
18          break
19        if  $|\pi[t][j_{curr}] - (\pi[t-1][i] + c[t][i][j_{curr}][1])| < \epsilon$ 
20          Add the sold out arc in  $\mathcal{G}_{sub}$  // Type 1
21           $j_{curr} \leftarrow i$ 
22           $arc\_found \leftarrow \text{True}$ 
23          break
24    if not  $arc\_found$ 
25      break
26 return  $\mathcal{G}_{sub}$ 

```

2.4. Limites de l'approche KCU

Bien que cette méthode semble prometteuse en alliant la puissance de la compilation des connaissances avec un algorithme plus léger pour la recherche du scénario optimal, KCU ne garantit pas la diversité des scénarios extraits, ce qui peut ralentir la convergence si plusieurs chemins optimaux existent et sont spatialement distincts.

3. KCUD (KC Dual à 2 chemins optimaux distants)

3.1. Introduction

Cette heuristique se base essentiellement sur KCU. Comme indiqué précédemment, KCU remonte le scénario pour lequel l'incertitude a été utilisée le plus tard possible dans la planification. Il s'agit d'un choix de programmation ; nous aurions tout aussi bien pu retourner le scénario selon lequel l'incertitude a été utilisée en totalité le plus tôt possible. Ce choix nous a conduits à nous interroger sur ce qu'il en serait si nous remontions ces deux types de scénarios à chaque itération. Cela correspondrait à une approche d'orthogonalité, car les deux chemins retournés seraient les scénarios optimaux les plus distants en termes de temporalité d'incertitude. Nous nous attendons à ce que le temps total de KCUD puisse être inférieur à celui de KCU et qu'il y ait moins d'itérations. L'algorithme reprend la structure de KCU ; il s'agit d'exploiter la structure de la matrice de programmation dynamique pour extraire non pas un, mais deux chemins distincts. Ce premier chemin est obtenu en privilégiant systématiquement les transitions issues des budgets les plus faibles (consommation tardive), tandis que le second privilégie les transitions issues des budgets élevés (consommation précoce).

Cependant, bien que cette méthode KCUD (pour *Dual*) soit prometteuse en termes de temps de calcul, il est important de rappeler quelques points :

- **La dégénérescence vers un chemin unique** : Il est tout à fait possible que la matrice de programmation dynamique ne contienne qu'un seul scénario optimal. Dans ce cas, les itérations ascendantes et descendantes renverront exactement le même scénario. L'effet d'orthogonalité de KCUD est alors nul ; cette méthode serait équivalente à la méthode KCU.
- **Orthogonalité vs Distance** : KCUD garantit formellement l'extraction des deux scénarios optimaux, sous l'hypothèse qu'il en existe au moins deux, les plus distants en termes de temporalité de consommation du budget. Néanmoins, cela ne garantit pas que les deux scénarios soient les plus distants mathématiquement au sens d'une distance L_1 .

3.2. Algorithme Glouton

L'Algorithme ?? a une complexité en $\mathcal{O}(T\Gamma)$. La recherche des extrema initiaux à $t = T$ prend $\mathcal{O}(\Gamma)$. Ensuite, l'algorithme fait appel à deux extractions gloutonnes (voir Algorithme ??), chacune s'exécutant en $\mathcal{O}(T \times \Gamma)$

Algorithm 2: Extraction of the optimal paths (KCUD)

Require: Dual matrix π , Costs matrix c , Optimal cost ub_cost , Tolerance ϵ
Ensure: Partial graph $\mathcal{G}_{sub}$ containing the two paths

```

1  $\mathcal{G}_{sub} \leftarrow \emptyset$ 
2  $start\_j\_late \leftarrow -1$ 
3  $start\_j\_early \leftarrow -1$ 
4 for  $i = 0$  to  $\Gamma$                                      // Searching for starting points at  $t = T$ 
5
6   if  $|\pi[T][i] - ub\_cost| < \epsilon$ 
7     if  $start\_j\_late == -1$ 
8        $start\_j\_late \leftarrow i$                                // First budget encountered (the lowest index)
9      $start\_j\_early \leftarrow i$                                // Last budget encountered (the highest index)
10
11 if  $start\_j\_late \neq -1$ 
12    $\mathcal{G}_{sub} \leftarrow \mathcal{G}_{sub} \cup \text{Adding the arches towards the well } (T + 1, 0)$ 
13    $\text{Extract\_Extreme\_Path}(start\_j\_late, \text{"ascendant"}, \mathcal{G}_{sub})$ 
14   if  $start\_j\_late \neq start\_j\_early$                        // Avoid recalculating if the points are =
15      $\text{Extract\_Extreme\_Path}(start\_j\_early, \text{"descendant"}, \mathcal{G}_{sub})$ 
16 return  $\mathcal{G}_{sub}$ 

```

Algorithm 3: Configurable reporting function

```

1 Function  $\text{Extract\_Extreme\_Path}(j_{start}, direction, \mathcal{G}_{sub})$ 
2    $j_{curr} \leftarrow j_{start}$ 
3   for  $t = T$  downto 1
4      $arc\_found \leftarrow \text{False}$ 
5     Define the loop according to  $direction : 0 \rightarrow j_{curr}$  or  $j_{curr} \rightarrow 0$ 
6     for each  $i$  in the loop
7       if The transition  $(i \rightarrow j_{curr})$  is physically valid
8         if  $|\pi[t][j_{curr}] - (\pi[t-1][i] + c[t][i][j_{curr}][0])| < \epsilon$ 
9            $\mathcal{G}_{sub} \leftarrow \mathcal{G}_{sub} \cup \{(t-1, i) \xrightarrow{0} (t, j_{curr})\}$            // Type 0
10           $j_{curr} \leftarrow i, arc\_found \leftarrow \text{True}$ 
11          break
12         if  $|\pi[t][j_{curr}] - (\pi[t-1][i] + c[t][i][j_{curr}][1])| < \epsilon$ 
13            $\mathcal{G}_{sub} \leftarrow \mathcal{G}_{sub} \cup \{(t-1, i) \xrightarrow{1} (t, j_{curr})\}$            // Type 1
14           $j_{curr} \leftarrow i, arc\_found \leftarrow \text{True}$ 
15          break
16   if not  $arc\_found$ 
17     break

```

3.3. Limites de l'approche KCUD

Intuitivement, l'heuristique KCUD vise à maximiser l'écart entre les scénarios extraits en forçant la consommation de l'incertitude aux deux extrêmes temporels. Il serait alors tentant de considérer KCUD comme un cas particulier de $KCHOG(N = 2)$, où N est le quota de scénarios à remonter. Or, d'un point de vue mathématique, ces deux méthodes diffèrent. KCUD garantit l'écartement temporel mais pas l'orthogonalité.

En effet, KCUD repose sur une logique gloutonne. Lors de la phase de *backtracking*, l'algorithme sélectionne à chaque étape t le budget i le plus faible (resp. le plus élevé) garantissant l'optimalité. Ce choix local peut contraindre le reste de la trajectoire en s'enfermant dans un goulot. KCHOG va d'abord générer des chemins complets depuis tous les nœuds atteignant ub_cost , et non pas le premier trouvé, puis va évaluer leurs distances.

L'algorithme identifie deux points de départ à l'horizon $t = T$: le budget minimal *start_j_late* et le budget maximal *start_j_early*. Cela implique qu'un seul nœud terminal atteint le coût optimal *ub_cost*. Dans ce cas, la condition de la ligne 13 empêche le recalcul du même chemin, et l'algorithme se réduit à une extraction unique (équivalente à KCU). Ce comportement évite la redondance inutile de scénarios dans le graphe partiel du Maître.

Comme indiqué précédemment, KCUD ne garantit pas le renvoi de deux scénarios optimaux distincts. Au vu des deux limites évoquées, nous pourrions considérer l'approche KCHOG (voir Section ??) comme une solution. De par sa construction, elle évite les goulots d'étranglement et permet également d'éviter le retour de deux mêmes scénarios en prenant, si besoin, des scénarios dans l'ensemble relâché.

4. KCRDKL (KC Random K Lexicographical)

4.1. Introduction

Bien que la méthode de compilation de connaissances soit performante pour des budgets d'incertitude limités, son temps de résolution se dégrade fortement lorsque l'espace d'incertitude Γ augmente. Cette perte d'efficacité s'explique par la densité du graphe partiel généré. Pour pallier ce problème d'explosion combinatoire, il est nécessaire de limiter strictement le nombre de scénarios adverses renvoyés au Maître à chaque itération.

L'approche KCRDKL (*Knowledge Compilation Random K Lexicographical*) a donc été introduite comme modèle de référence. Sa structure algorithmique est identique à celle de KCHOGL (que nous définirons par la suite dans la Section ??), mais le critère de sélection spatial est remplacé par un tirage aléatoire uniforme. La comparaison des performances entre KCHOGL et KCRDKL permet de mesurer le gain apporté par la diversité spatiale des scénarios.

4.2. Algorithme d'Extraction Aléatoire Priorisée

Pour que la comparaison avec l'approche KCHOGL (KCHOG *Lexicographical*, voir Section ??) soit rigoureuse, KCRDKL doit adopter la même structure de priorisation². L'algorithme cherche en priorité à saturer le quota N avec des pires scénarios absolus. Ce n'est que si le nombre de pires scénarios est inférieur à N que l'algorithme vient piocher dans l'ensemble relaxé pour compléter sa sélection. La fonction *Calculate_Bound()* calcule la borne minimale pour considérer un chemin dans l'ensemble relaxé des chemins critiques. Cette borne vaut $\tau \times ub_cost$, où ub_cost est le coût du pire chemin et $\tau \in [0, 1]$. Si $\tau = 1$, alors nous ne considérons que les chemins de coût ub_cost , donc les chemins optimaux.

L'Algorithme ?? a une complexité en $\mathcal{O}(LTT)$, car il est notamment dominé par l'exploration du DFS. La fonction stochastique *Shuffle* prend un temps négligeable en $\mathcal{O}(K)$.

2. Initialement, une heuristique nommée KCRDK pour *Knowledge Compilation Random K* avait été introduite pour servir de baseline à KCHOG. Cependant, ayant amélioré KCHOG en KCHOGL, il a été nécessaire de reproduire la transition sur KCRDK, mais dans un souci de simplicité, nous n'avons décrit que l'heuristique privilégiée.

Algorithm 4: Extraction of N scenarios via prioritized random sampling (KCRDKL)

Require: Dual matrix π , Costs matrix c , Optimal cost ub_cost , Limit on scenarios to push up N , Exploration limit L , Tolerance ϵ

Ensure: Set of scenarios P_{sel} (max size N)

```

1  $sub\_OPT \leftarrow \text{Calculate\_Bound}(ub\_cost, \tau)$ 
2  $P_{opt} \leftarrow \emptyset$  // Set of optimal scenarios
3  $P_{cand} \leftarrow \emptyset$  // Set of sub-optimal scenarios
4  $P_{sel} \leftarrow \emptyset$ 
5 for  $i = 0$  to  $\Gamma$  // Phase 1: Extraction of optimal scenarios
6
7   if  $|\pi[T][i] - ub\_cost| < \epsilon$ 
8      $\text{Extract\_Paths\_DFS}(T, i, \pi, c, P_{opt}, L, \epsilon)$ 
9   if  $P_{opt} \neq \emptyset$ 
10      $P_{sel} \leftarrow P_{sel} \cup \{P_{opt}[0]\}$  // Convergence guarantee
11      $P_{opt} \leftarrow P_{opt} \setminus \{P_{opt}[0]\}$ 
12      $P_{opt} \leftarrow \text{Shuffle}(P_{opt})$ 
13     while  $|P_{sel}| < N$  and  $P_{opt} \neq \emptyset$  // Priority filling using optimal values
14        $\text{Transfer the first element of } P_{opt} \text{ to } P_{sel}$ 
15
16 if  $|P_{sel}| < N$  // Phase 2: Relaxed extraction if  $N$  is not reached
17
18   for  $i = 0$  to  $\Gamma$ 
19     if  $\pi[T][i] \geq sub\_OPT$  and  $|\pi[T][i] - ub\_cost| \geq \epsilon$ 
20        $\text{Extract\_Paths\_DFS}(T, i, \pi, c, P_{cand}, L, \epsilon)$ 
21    $P_{cand} \leftarrow \text{Shuffle}(P_{cand})$ 
22   while  $|P_{sel}| < N$  and  $P_{cand} \neq \emptyset$ 
23      $\text{Transfer the first element of } P_{cand} \text{ to } P_{sel}$ 
24 return  $P_{sel}$ 

```

4.3. Limites de l'approche KCRDKL

L'implémentation de KCRDKL suit une logique lexicographique : **1.** maximiser le coût des attaques, **2.** diversifier l'information de manière stochastique.

Pour comparer KCHOGL et KCRDKL, si, face à un grand nombre d'attaques adverses strictement équivalentes en termes de coût, KCHOGL permet au problème Maître de converger en un nombre d'itérations significativement inférieur à KCRDKL. Nous prouvons que la topologie spatiale des scénarios porte une information importante pour la résolution du problème de *Lot-Sizing* robuste. La baseline aléatoire justifie donc l'effort computationnel du calcul des distances.

5. KCHOG (KC OrthoGonality)

5.1. Introduction

Afin d'accélérer la convergence de la méthode de Benders Adverse tout en évitant de saturer le problème maître avec des coupes redondantes, nous avons développé une Heuristique d'OrthoGonalité (KCHOG). Son objectif est d'extraire un sous-ensemble de scénarios adverses qui soient à la fois très pénalisants et spatialement diversifiés.

Cette heuristique s'articule autour de deux composants algorithmiques appelés dans le Sous-Problème :

1. Une phase d'extraction (voir Section ??) s'appuie sur un algorithme de recherche en profondeur (DFS) déterministe pour parcourir le graphe des états de la programmation dynamique et remonter les chemins critiques.
2. Une phase de filtrage et de sélection (voir Sections ?? et ??) qui s'appuie sur le calcul de la distance de Manhattan pour identifier un sous-ensemble de chemins maximisant la dispersion spatiale par rapport à l'espace d'incertitude (approche **MaxMin**).

L'idée algorithmique de l'heuristique d'orthogonalité est la suivante :

1. La passe *forward* fournit la matrice des coûts π , et remonte le pire coût que nous noterons ub_cost .
2. Nous lançons une première fois le DFS à partir du nœud qui a mené à ub_cost . Cette étape permettra de trouver tous les chemins optimaux, qui seront stockés dans la variable $P_{opt}[]$.
3. Nous construisons ensuite l'ensemble relaxé $P_{cand}[]$ correspondant à $\tau \times ub_cost$. Tous les nœuds finaux (T, j) ayant un coût supérieur ou égal deviennent des nœuds critiques et font partie de cet ensemble.
4. Nous relançons le DFS à partir de ces nœuds. Tous les chemins remontés font partie de l'ensemble relaxé³.
5. Enfin, nous prenons le premier chemin optimal trouvé ($P_{opt}[0]$) et appliquons la discrétisation par la distance L_1 sur l'ensemble des chemins contenus dans $P_{cand}[]$.
Note : Afin d'empêcher les doublons, l'ensemble relaxé supprime par défaut $P_{opt}[0]$.
6. Comme nous le verrons ci-dessous, l'optimalité est garantie car nous ajoutons au moins un chemin optimal dans la matrice remontée au Maître. Cependant, nous avons fait le compromis de préférer potentiellement un chemin sous optimal distant plutôt qu'un chemin optimal non distant.

Une alternative, présentée à la Section ?? consiste à remonter les N chemins les plus distants dans l'ensemble des chemins optimaux, et si N n'est pas atteint, alors nous effectuons la recherche sur l'ensemble relaxé. Cela permettra d'assurer davantage de chemins optimaux.

Note : KCHOG($N = 1, L = 1$), où L est la limite d'exploration du DFS, correspond globalement à l'approche KCU détaillée dans la Section ??.

5.2. Algorithme de Recherche en Profondeur (DFS)

Bien que le coût maximal adverse absolu à l'horizon T soit connu, les séquences de décisions qui y mènent doivent être reconstruites. Le DFS procède donc à un *backtracking* déterministe depuis les nœuds terminaux jusqu'à l'origine du graphe. La mécanique de remontée repose sur les principes suivants :

3. L'ensemble relaxé contient donc les chemins optimaux et sous-optimaux.

- **Exploration des antécédents (l. 9)** : Depuis un état courant (t, j) , l'algorithme évalue tous les budgets i ⁴ potentiellement consommés à la période précédente $(t - 1)$ ⁵.
- **Faisabilité des transitions (l. 10 & 11)** : L'algorithme s'assure que la variation de budget $(j - i)$ respecte le budget d'incertitude local Δt_{t-1} . De plus, dans le cas d'un sur-stockage (arc de type 0), l'attaque de l'adversaire réduisant la demande nominale ne doit pas conduire à une demande effective négative, cela est imposé par $(j - i) \leq sol.dt[t - 1]$.
- **Troncature (l. 1)** : Le parcours étant réalisé sur un DAG combinatoire, le nombre de pires scénarios peut exploser. Une limite L est imposée pour stopper l'extraction. Par conséquent, bien que l'algorithme soit purement déterministe et sans doublon, il ne garantit pas l'exhaustivité de l'extraction si l'espace des chemins optimaux dépasse L .

L'Algorithme ?? a une complexité en $\mathcal{O}(LTT)$. Il s'agit d'un parcours en profondeur sur un DAG. La troncature limite le nombre de feuilles explorées à L . Chaque chemin ayant une longueur maximale T , et l'exploration des budgets i étant bornée par Γ , la complexité est restreinte par la limite L .

Notations : $sol.\Delta t[]$ correspond à la borne locale d'incertitude, tandis que $sol.dt[]$ est la demande nominale.

Algorithm 5: Recursive extraction of scenarios (DFS) (KCHOG)

Require: Period t , Local budget j , Dual matrix π , Costs matrix c , Nominale solution sol ,
Current path P_{curr} , Set of scenarios P_{all} , Exploration limit L , Tolerance ϵ

Ensure: P_{all} updated

```

1 if  $|P_{all}| \geq L$ 
2   return                                     // Truncation of the search space
3 if  $t == 0$ 
4    $P_{rev} \leftarrow \text{reverse}(P_{curr})$                                      // Righting
5    $P_{all} \leftarrow P_{all} \cup \{P_{rev}\}$ 
6   return
7 for  $i = 0$  to  $j$                                      // Assessment of past budgets
8
9   if  $j \leq i + sol.\Delta t[t - 1]$  and  $(t \neq 1 \text{ or } i == 0)$ 
10    if  $(j - i) \leq sol.dt[t - 1]$                                      // Type 0
11
12     if  $|\pi[t][j] - (\pi[t - 1][i] + c[t][i][j][0])| < \epsilon$ 
13       $P_{curr} \leftarrow P_{curr} \cup \{(t, i, j, 0)\}$ 
14       $\text{Extract\_Paths\_DFS}(t - 1, i, \pi, c, sol, P_{curr}, P_{all}, L, \epsilon)$ 
15       $P_{curr} \leftarrow P_{curr} \setminus \{(t, i, j, 0)\}$ 
16
17    if  $|\pi[t][j] - (\pi[t - 1][i] + c[t][i][j][1])| < \epsilon$                                      // Type 1
18
19      $P_{curr} \leftarrow P_{curr} \cup \{(t, i, j, 1)\}$ 
20      $\text{Extract\_Paths\_DFS}(t - 1, i, \pi, c, sol, P_{curr}, P_{all}, L, \epsilon)$ 
21      $P_{curr} \leftarrow P_{curr} \setminus \{(t, i, j, 1)\}$ 

```

4. Un compromis algorithmique a été fait lors de l'évaluation des nœuds à une période t . Nous partons du nœud le plus bas dans le graphe de budget.

5. La nature cumulative du budget impose que $0 \leq i \leq j$.

5.3. Algorithme de Calcul de Distance

Afin de quantifier la distance entre les trajectoires d'incertitude générées par l'adversaire, nous utilisons la distance de Manhattan. La norme L_1 est choisie car elle reflète la somme des écarts absolus de consommation du budget. Chaque chemin est encodé mathématiquement par la séquence des variations de budget incertain consommées à chaque période.

- Pour une période k donnée, un scénario induit une déviation locale $\Delta_i = i[k].j - i[k].i$, où $\Delta_i \in \mathbb{N}$ et $i \in \{A, B\}$;
- La divergence locale entre deux scénarios A et B est donnée par la différence absolue de ces déviations : $|\Delta_A - \Delta_B|$;
- La distance totale est la somme de ces divergences sur l'horizon de planification T .

Cette distance permet d'identifier l'orthogonalité des scénarios : plus elle est élevée, plus l'adversaire a attaqué des périodes différentes ou a utilisé son budget dans des directions opposées.

La complexité de l'Algorithme ?? est en $\mathcal{O}(T)$ car il s'agit d'un parcours linéaire simple des tableaux de taille T .

Algorithm 6: Calculate Manhattan distance (KCHOG)

```

Require: Path  $A$ , Path  $B$ 
Ensure: Distance  $d_{L1} \geq 0$ 
1 if  $|A| \neq |B|$ 
2   return 0
3  $sum\_diff \leftarrow 0$ 
4 for  $k = 1$  to  $|A|$ 
5    $\Delta_A \leftarrow A[k].j - A[k].i$  // Marginal budget consumed by A at  $k$ 
6    $\Delta_B \leftarrow B[k].j - B[k].i$ 
7    $sum\_diff \leftarrow sum\_diff + |\Delta_A - \Delta_B|$ 
8 return  $sum\_diff$ 

```

5.4. Algorithme de Sélection MaxMin

Une fois la matrice des chemins candidats générée par le DFS, nous implémentons un algorithme glouton de dispersion de type **MaxMin** pour sélectionner un sous-ensemble restreint de N scénarios. Contrairement au problème global qui est un **MinMax** sur les coûts, l'objectif est ici de **Maximiser** la distance **Minimale** entre les scénarios retenus afin de diversifier au maximum les informations renvoyées au Maître. Ce sous-ensemble sera agrégé dans le graphe matriciel renvoyé au problème maître.

La procédure est itérative :

- **Évaluation de la distance (Boucle Min, l. 1)** : Pour chaque chemin candidat de l'ensemble relaxé P_{cand} , nous calculons sa distance par rapport à l'ensemble des chemins déjà sélectionnés (P_{sel}). La plus petite de ces distances (min_dist) définit son degré d'isolement : une valeur faible indique une forte redondance avec un scénario déjà connu du maître.
- **Sélection du candidat (Boucle Max, l. 11)** : L'algorithme retient le candidat possédant le degré d'isolement min_dist le plus élevé. Ce scénario, considéré comme le plus orthogonal à la base de connaissances actuelle, est concaténé dans l'ensemble P_{sel} .

La complexité de l'Algorithme ?? est en $\mathcal{O}(LN^2T)$ ⁶. La boucle *while* principale tourne N fois. À chaque itération, nous parcourons les L candidats de l'ensemble relaxé. Pour chaque candidat, nous calculons la distance par rapport aux scénarios déjà sélectionnés (au pire N), ce qui coûte $\mathcal{O}(T)$. Total : $\sum_{s=1}^N (L \times s \times T) = \mathcal{O}(L \cdot N^2 \cdot T)$.

Algorithm 7: MaxMin Selection (KCHOG)

Require: Current selection P_{sel} , Candidate pool $Pool_{cand}$, Limit on scenarios N

Ensure: P_{sel} updated with orthogonal scenarios from $Pool_{cand}$

```

1 while  $|P_{sel}| < N$  and  $Pool_{cand} \neq \emptyset$ 
2    $best\_max\_min \leftarrow -1$ 
3    $best\_candidate \leftarrow \text{null}$ 
4   for each  $p_c \in Pool_{cand}$                                 // Calculate the distance of the sub-set
5
6      $min\_dist \leftarrow \infty$ 
7     for each  $p_s \in P_{sel}$ 
8        $dist \leftarrow \text{Manhattan\_Distance}(p_c, p_s)$ 
9       if  $dist < min\_dist$ 
10         $min\_dist \leftarrow dist$ 
11
12     if  $min\_dist > best\_max\_min$                             // Selection of the most distant candidate
13
14        $best\_max\_min \leftarrow min\_dist$ 
15        $best\_candidate \leftarrow p_c$ 
16
17    $P_{sel} \leftarrow P_{sel} \cup \{best\_candidate\}$ 
18    $Pool_{cand} \leftarrow Pool_{cand} \setminus \{best\_candidate\}$ 
19 return  $P_{sel}$ 

```

5.5. Limites de l'implémentation KCHOG

Comme présenté en introduction, il est important de souligner le compromis réalisé dans l'implémentation actuelle de l'heuristique KCHOG. Afin d'assurer la convergence mathématique du problème maître, l'algorithme garantit l'inclusion d'au moins un chemin critique absolu (le chemin $P_{opt}[0]$ de coût ub_cost). Cependant, lors des itérations suivantes de la boucle **MaxMin**, la fonction de sélection s'appuie exclusivement sur la distance spatiale sans tenir compte du coût dual des chemins candidats.

6. Il est possible de réduire sa complexité à $\mathcal{O}(LNT)$ en mémorisant les distances.

Par conséquent, l'algorithme privilégiera un scénario relaxé (sous-optimal) s'il est très éloigné de la sélection actuelle, écartant potentiellement d'autres scénarios optimaux dont le coût d'attaque pour le maître serait pourtant supérieur. Une évolution intéressante consisterait à appliquer la dispersion spatiale **MaxMin** en priorité sur l'ensemble P_{opt} , avant d'élargir la recherche à l'espace P_{cand} si la diversité des pires cas est insuffisante pour atteindre le quota N .

6. KCHOGL (KCHOG Lexicographical)

6.1. Introduction

Cette méthode tient compte du compromis fait dans KCHOG. C'est-à-dire que nous priorisons la sévérité d'un scénario optimal par rapport à la distance d'un scénario sous-optimal. L'algorithme ?? est basé sur celui de KCHOG (voir Algorithme ??). Si N est le nombre de scénarios à remonter au Maître, nous allons d'abord discrétiser les scénarios optimaux de coût ub_cost . Puis, si le quota N n'est pas atteint, nous discrétisons par la distance L_1 dans l'ensemble relaxé contenant uniquement des scénarios de coût $[sub_OPT, ub_cost]$.

6.2. Algorithme d'Extraction Aléatoire Priorisé

L'Algorithme ?? a une complexité importante en $\mathcal{O}(LTT + LN^2T)$ car il combine la complexité du DFS ainsi que celle de l'Algorithme ?? pour le filtrage spécial.

Algorithm 8: Extraction of N scenarios via L_1 -distance (KCHOGL)

Require: Dual matrix π , Cost matrix c , Optimal cost ub_cost , Limit on scenarios to push up N , Exploration limit L , Tolerance ϵ

Ensure: Set of scenarios P_{sel} (of maximum size N)

```

1  $sub\_OPT \leftarrow \text{Calculate\_Bound}(ub\_cost, \tau)$ 
2  $P_{opt} \leftarrow \emptyset$  // Set of optimal scenarios
3  $P_{cand} \leftarrow \emptyset$  // Set of sub-optimal scenarios
4  $P_{sel} \leftarrow \emptyset$ 
5 for  $i = 0$  to  $\Gamma$  // Phase 1: Extraction of optimal scenarios
6
7   if  $|\pi[T][i] - ub\_cost| < \epsilon$ 
8   └  $\text{Extract\_Paths\_DFS}(T, i, \pi, c, P_{opt}, L, \epsilon)$ 
9 if  $P_{opt} \neq \emptyset$ 
10    $P_{sel} \leftarrow P_{sel} \cup \{P_{opt}[0]\}$  // Convergence guarantee
11    $P_{opt} \leftarrow P_{opt} \setminus \{P_{opt}[0]\}$ 
12   // Spatial filtering among remaining optimal scenarios
13    $P_{sel} \leftarrow \text{MaxMin\_Selection}(P_{sel}, P_{opt}, N)$  // See Algorithm ??
14
15 if  $|P_{sel}| < N$  // Phase 2: Relaxed extraction if  $N$  is not reached
16
17   for  $i = 0$  to  $\Gamma$ 
18   └ if  $\pi[T][i] \geq sub\_OPT$  and  $|\pi[T][i] - ub\_cost| \geq \epsilon$ 
19   └  $\text{Extract\_Paths\_DFS}(T, i, \pi, c, P_{cand}, L, \epsilon)$ 
20   // Completion with sub-optimal scenarios
21    $P_{sel} \leftarrow \text{MaxMin\_Selection}(P_{sel}, P_{cand}, N)$ 
22 return  $P_{sel}$ 

```

6.3. Limites de l'approche KCHOGL

Comme dit dans la Section ??, cette approche est comparable à KCRDKL. Nous nous attendons à ce que KCHOGL ait de meilleures performances lorsque le quota N reste relativement faible. Au-delà, une approche aléatoire devrait prendre moins de temps qu'un calcul de distance.

7. Expérimentations

La logique pour le plan d'expérience est la suivante. La problématique est de prouver que la compilation de connaissances, regroupant toutes les méthodes dites KC, est plus performante que la méthode classique de Benders Adverse. Une fois cela fait, nous devons encore trouver l'heuristique de factorisation la plus performante pour chaque type de configuration. Nous avons donc effectué trois expérimentations principales ($N = 1, N = 2, N > 2$). La première sert à prouver l'apport de la compilation de connaissances en comparant BA et KCU. Ces deux méthodes renvoient un scénario à la fois, ce qui permet d'isoler les performances de la factorisation matricielle. La deuxième expérience permet de différencier les distances temporelles et spatiales des approches KCUD et KCHOGL. Nous déterminerons dans la troisième si l'utilisation de l'orthogonalité est préférable au regard de l'heuristique aléatoire. Enfin, les expériences 4 et 5 permettront de comparer respectivement les performances de BAE et l'influence de la périodicité sur les heuristiques susmentionnées.

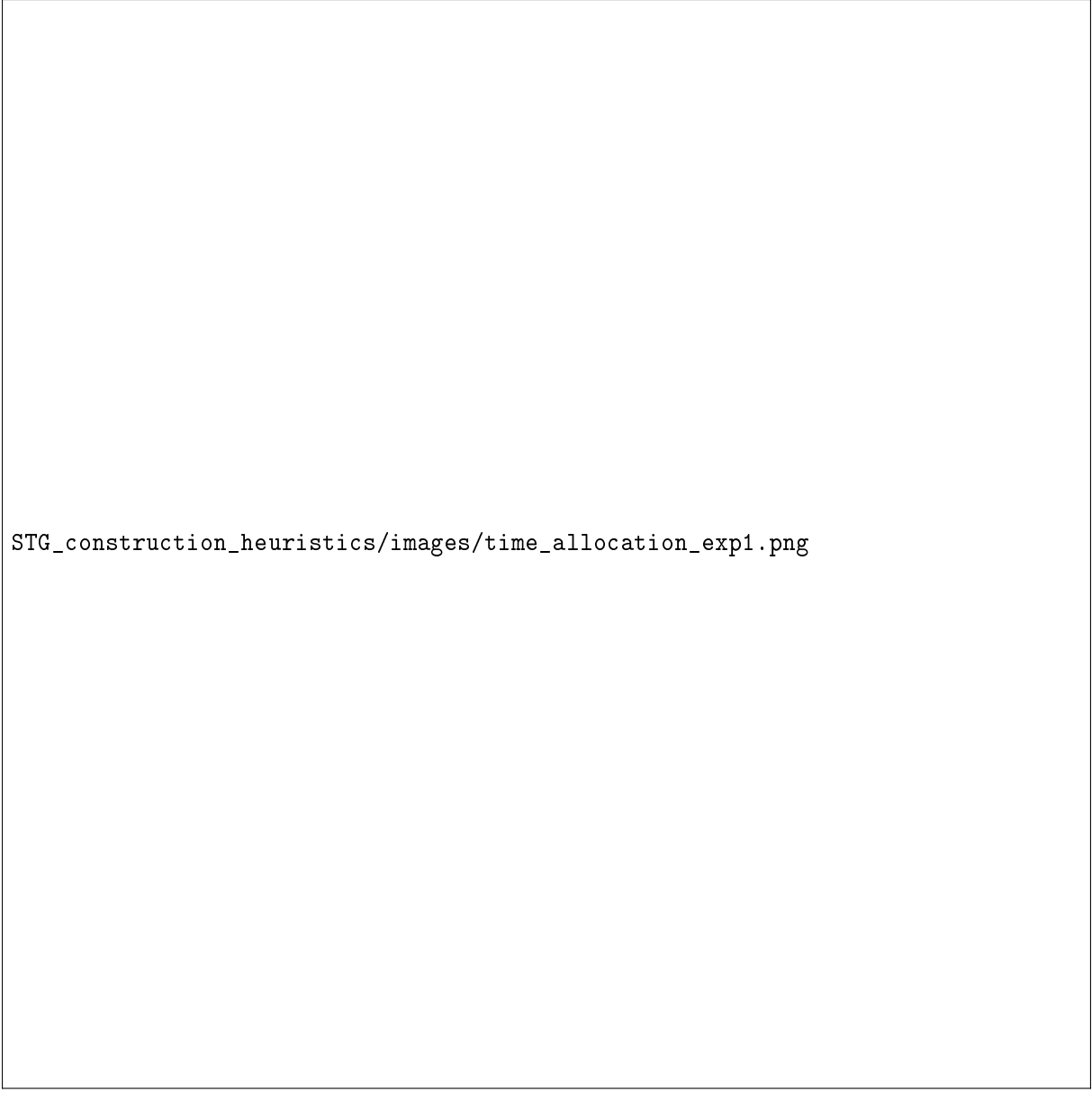
Tous les tests ont été réalisés sur le même ensemble d'instances. Cet ensemble est constitué de 50 instances de $T = 52$ périodes représentant une année de production. Une périodicité trimestrielle de 13 semaines a été mise en place. Pour chaque période $t \leq T$, 20 références ont dû être planifiées. La probabilité qu'une référence j soit demandée à un instant t , durant une période dense, suit une loi de Bernoulli de paramètre $p_{demand} = 0,7$. La probabilité pour une période creuse est égale à $(1 - p_{demand})$. Le budget d'incertitude Γ prend ses valeurs dans son ensemble de définition $\mathcal{D}_\Gamma = \{1, 11, \dots, 91, 101\}$. Pour les méthodes implémentant l'heuristique *SelectFew*, la probabilité de sélection d'un arc suit également une loi de Bernoulli de paramètre $p_{SelectFew} = 0.35$ ⁷. Les calculs ont été réalisés sur la version 22.1 du solveur CPLEX. Notre problème est modélisé en *C++*.

7.1. Expérience 1 : apports de la compilation de connaissances ($N = 1$)

L'objectif de cette première expérience est d'isoler l'apport structurel de la factorisation matricielle. Pour ce faire, nous comparons la méthode de Benders Adverse (BA) avec notre méthode KCU (*Knowledge Compilation Unique*). Ces deux méthodes sont étudiées sur le même pied d'égalité quant au volume d'information transmis. A chaque itération, le Sous-Problème ne renvoie qu'un unique scénario optimal.

L'enjeu est de déterminer si l'intégration de ce scénario unique sous forme de chemin dans un graphe orienté acyclique permet au problème Maître de converger plus efficacement que l'ajout d'une coupe de Benders standard constituée de contraintes linéaires. Les tests sont réalisés en faisant varier le budget maximal d'incertitude Γ dans son ensemble $\mathcal{D}_\Gamma$.

7. Nous rappelons tout de même que si, pour un nœud i , aucun arc n'est choisi par cette heuristique, le dernier vérifié est sélectionné afin de garantir une solution. De plus, ce paramètre a été fixé empiriquement.



STG_construction_heuristics/images/time_allocation_exp1.png

FIGURE 1 – Comparison of sub-problem and master problem times for BA & KCU.



STG_construction_heuristics/images/exp1_comparative_analysis_BA_KCU.png

FIGURE 2 – Comparative analysis between BA & KCU.

L'analyse des performances, illustrée par les Figures ?? et ??, met en évidence une certaine supériorité de l'approche KCU par rapport à nos trois métriques. Premièrement, concernant le nombre d'itérations, la méthode BA voit son nombre d'itérations croître rapidement pour plafonner autour de 7,5 itérations en moyenne dès que $\Gamma \geq 20$. À l'inverse, l'approche KCU converge systématiquement pour environ 1,5 itérations, et ce, quel que soit le budget d'incertitude.

La diminution du nombre d'itérations se traduit par un gain de temps relativement significatif. Le premier graphique de la Figure ?? montre qu'en moyenne, KCU domine systématiquement BA. Nous observons un pic d'efficacité autour de $\Gamma = 11$, où l'approche matricielle KCU résout le problème en moyenne 12 fois plus vite. Pour des budgets d'incertitudes plus importants, le ratio se stabilise, KCU restant environ 2 à 3 fois plus rapide. Le gain de temps moyen brut croît proportionnellement à Γ , atteignant plus de 0,31 seconde par instance pour $\Gamma = 100$.

Le graphique ?? confirme la robustesse de la formulation du problème Maître de KCU. Pour la méthode BA, le temps cumulé alloué au Maître (en rouge) augmente lorsque Γ croît. Cela vient de la difficulté du solveur à gérer l'accumulation de contraintes linéaires au fil des itérations. Pour KCU (qui est classé au premier rang selon quatre heuristiques testées, avec 4,805s cumulées

contre 16,930s pour BA), le temps du Maître reste relativement faible. C’est la preuve que nous cherchions, selon laquelle la factorisation par approche matricielle est beaucoup plus digeste et compacte pour *CPLEX*.

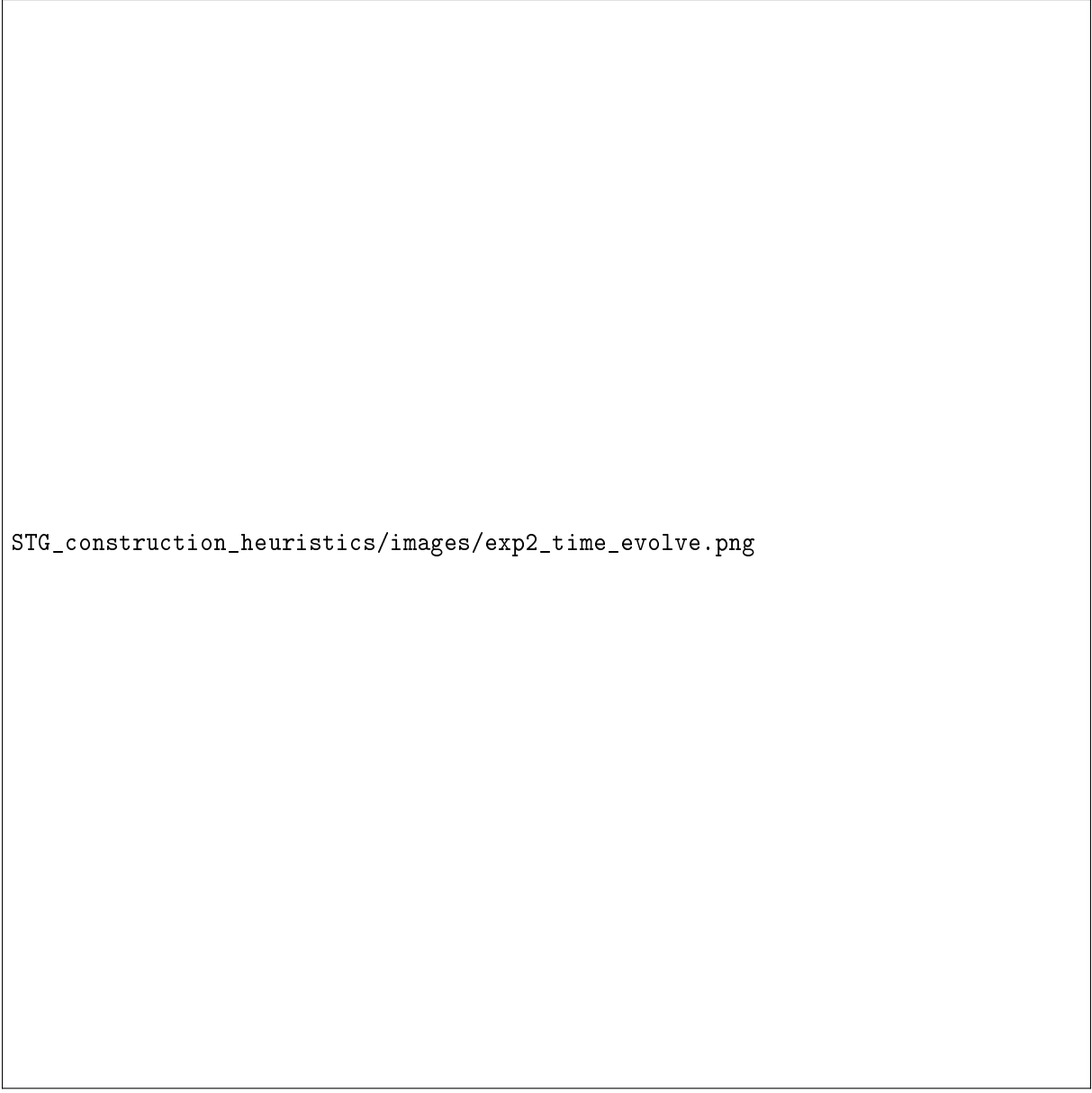
Enfin, cette première expérience valide la conjecture de base de notre modèle. A information extraite égale ($N = 1$), l’encodage de la connaissance incertaine sous forme de graphe offre une relaxation nettement plus forte au problème Maître qu’une coupe classique. KCU divise par 5 le nombre d’itérations nécessaires à la convergence. Maintenant que nous avons prouvé la supériorité de cette méthode, nous pouvons désormais l’exploiter sur des heuristiques plus performantes.

7.2. Expérience 2 : extremum temporel vs. extremum spatial ($N = 2$)

Cette deuxième expérience a pour but, lorsque $N = 2$, d’évaluer quelle définition de la diversité génère les coupes les plus efficaces. D’un côté, nous avons l’approche KCHOGL qui sélectionne les deux chemins critiques maximisant la distance L_1 . De l’autre, nous avons l’heuristique KCUD qui sélectionne deux chemins optimaux maximisant l’écart de période de consommation du budget d’incertitude. Nous conjecturons que l’approche KCUD sera meilleure sur le temps moyen global de résolution. D’une part, son algorithme d’extraction glouton possède une complexité inférieure à celle d’une exploration en profondeur. D’autre part, la nature du problème de *Lot-Sizing* suggère que forcer les extrêmes temporels sera plus efficace pour encadrer la solution optimale. L’heuristique aléatoire KCRDKL($N = 2$) est conservée comme *baseline* stochastique.

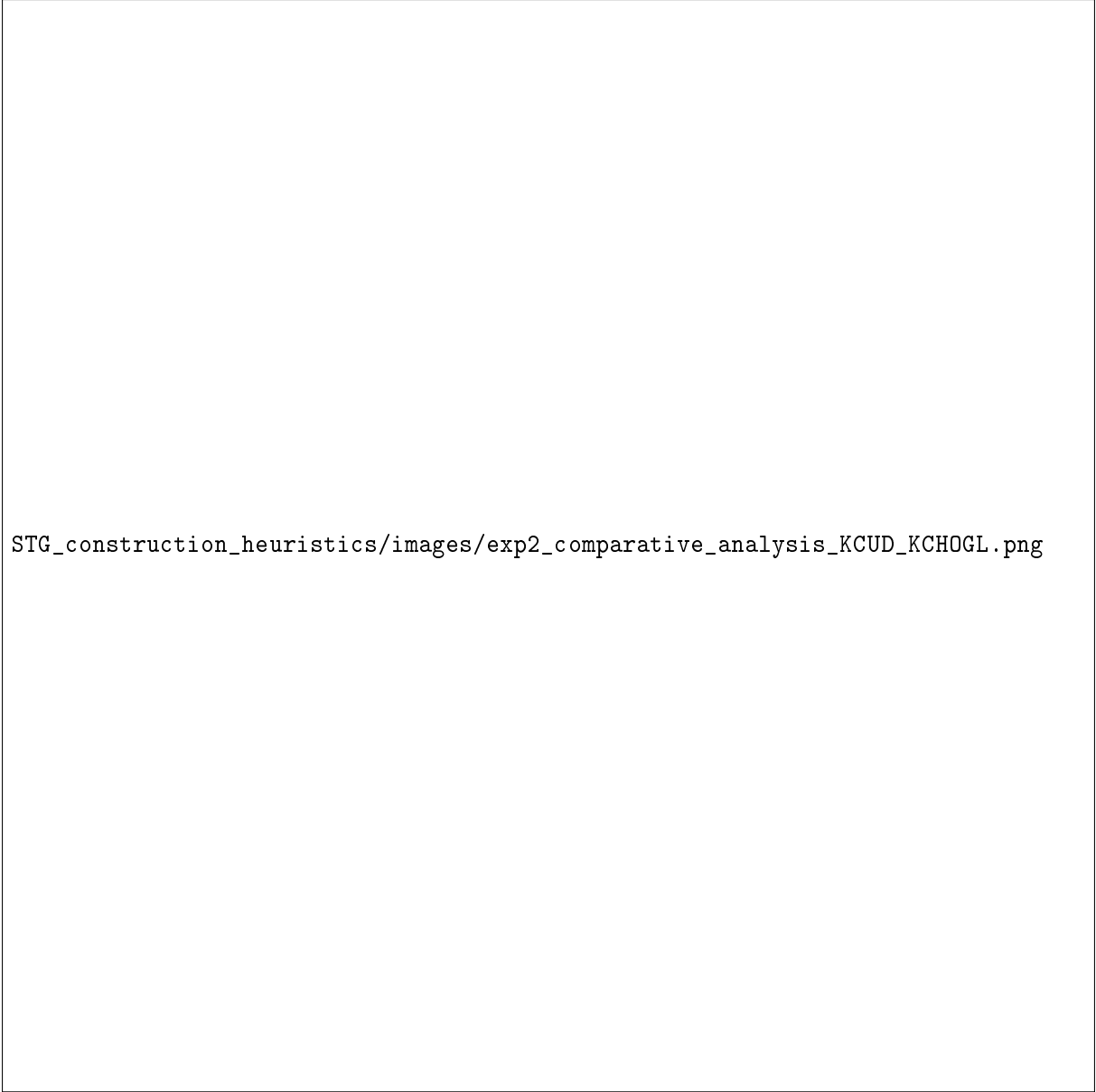
L’expérience se fait sur les mêmes instances de test. Le paramètre de sélection est fixé à $N = 2$. En prenant $\tau = 1$, les deux méthodes sont forcées de piocher des scénarios dans l’ensemble optimal. La limite d’exploration du DFS, qui n’affecte que KCHOGL et KCRDKL, varie dans l’ensemble $\mathcal{D}_L = \{100, 200, \dots, 1500\}$.

Les résultats expérimentaux confirment notre conjecture. KCUD est la méthode la plus performante.



STG_construction_heuristics/images/exp2_time_evolve.png

FIGURE 3 – Comparison of sub-problem and master problem times for KCUD, KCHOGL & KCRDKL.



STG_construction_heuristics/images/exp2_comparative_analysis_KCUD_KCHOGL.png

FIGURE 4 – Comparative analysis between KCUD & KCHOGL.

L'analyse des temps cumulatifs moyens via la Figure ?? nous permet d'établir un classement des heuristiques. Nous avons : **1.** KCUD ($\approx 49,58$ s), **2.** KCHOGL ($\approx 54,15$ s), **3.** KCRDKL ($\approx 55,58$ s). De plus, la Figure ?? nous montre que la majorité des ratios se situent dans la zone de victoire de KCUD. Comme anticipé, la complexité gloutonne de KCUD, en $\mathcal{O}(TT)$, lui permet d'être plus rapide que la seconde méthode. Or, sa complexité n'est pas son seul point fort, puisque KCUD diminue également le temps de résolution du Maître ainsi que son nombre d'itérations. Le graphique croisé (voir *Impact of Gamma on averaged iterations*, Figure ??) montre que KCUD effectue moins d'itérations.

Cette performance s'explique par la structure même du problème de *Lot-Sizing*. Contrairement à une distance spatiale L_1 qui mesure l'écart moyen sur l'horizon, la stratégie de KCUD force l'adversaire à tester les extrêmes temporels de consommation du budget d'incertitude. Dans un contexte de planification de production, ces périodes sont souvent critiques pour la gestion des stocks et des coûts de mise en route. La diversité temporelle génère ainsi des coupes plus discriminantes pour le Maître que la diversité géométrique, car elle explore les configurations les plus contraignantes de la chaîne de production. KCUD offre à la fois l'algorithme d'extraction le

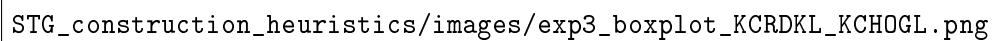
plus léger et les coupes contenant le plus d'informations, ce qui rend inutiles les autres approches envisagées.

7.3. Expérience 3 : apports de la distance ($N > 2$)

La conjecture que nous avons faite en développant l'heuristique d'orthogonalité (KCHOGL) était qu'en sélectionnant, parmi les pires scénarios, ceux qui maximisent la distance de Manhattan, nous fournissons au problème Maître des coupes spatialement diversifiées. L'hypothèse sous-jacente était que cette diversité d'information permettrait au Maître de converger en un nombre réduit d'itérations. Le compromis théorique que nous avons fait était qu'il fallait accepter une augmentation du temps de résolution du Sous-Problème (due notamment à la complexité quadratique du calcul des distances), mais qu'elle serait compensée par un gain de temps global grâce à la réduction des itérations du Maître.

A l'inverse, l'heuristique KCRDKL ne perd pas de temps dans le calcul des distances. Nous conjecturons donc que le temps de son Sous-Problème restera stable et indépendant de l'augmentation des différents paramètres, bien que l'aspect aléatoire des coupes puisse théoriquement engendrer plus d'itérations pour converger.

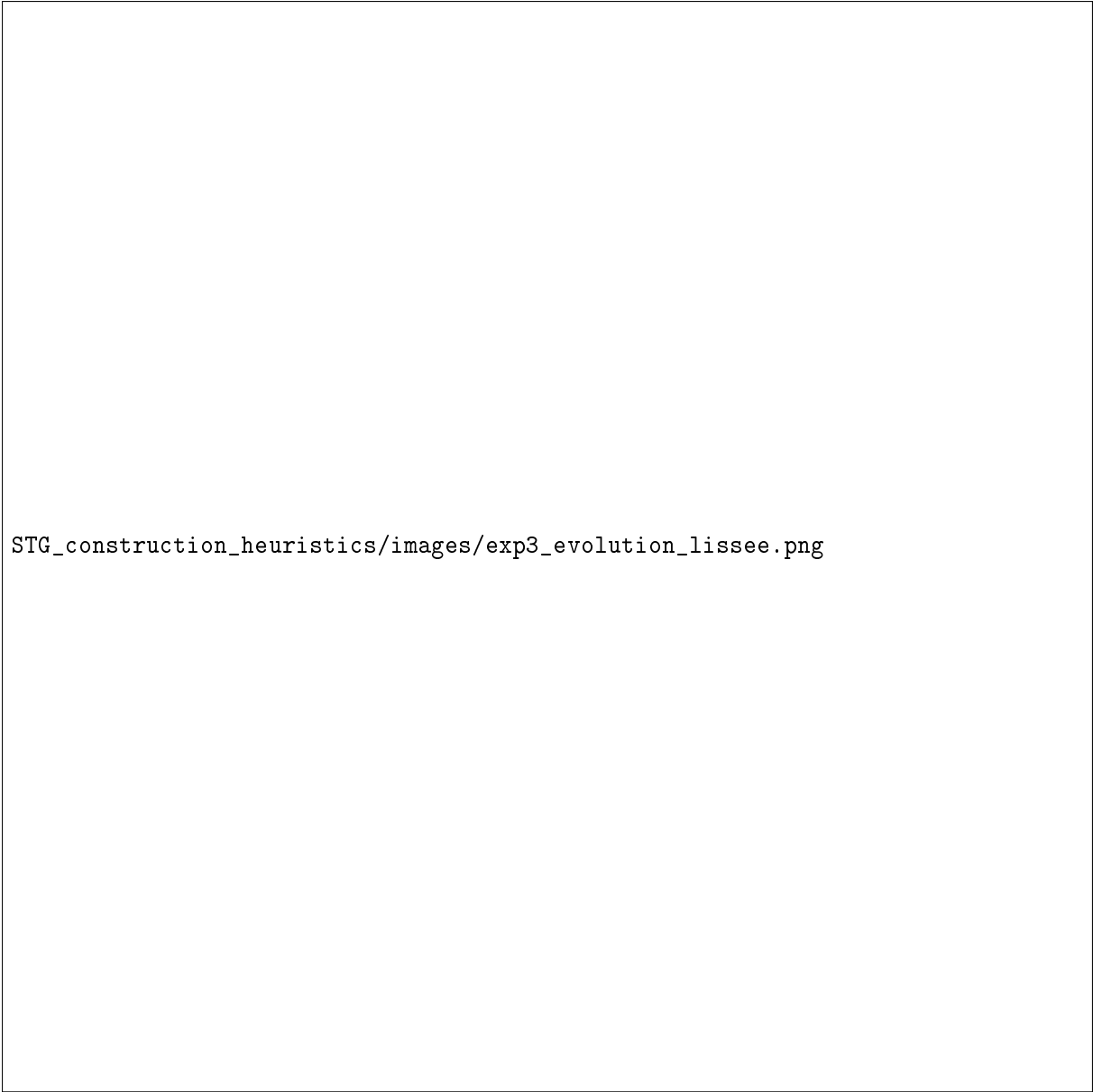
Pour réaliser l'expérience, nous avons pris le même jeu de données en faisant varier les paramètres. Le nombre de chemins remontés au Maître varie dans l'ensemble $\mathcal{D}_N = \{3, 5, 7, 9, 11\}$, la limite d'exploration de l'algorithme DFS appartient à $\mathcal{D}_L = \{100, 200, \dots, 1500\}$, et le seuil de relaxation τ varie dans $\mathcal{D}_\tau = \{0.8, 0.9, 1\}$.



STG_construction_heuristics/images/exp3_boxplot_KCRDKL_KCHOGL.png

FIGURE 5 – Speedup distribution between KCRDKL & KCHOGL.

L'analyse, Figure ??, de la distribution du ratio des temps moyens de résolution globaux met en évidence que la méthode aléatoire KCRDKL est globalement plus rapide que KCHOGL. La majorité des ratios se situe en dessous du seuil d'égalité. Cela confirme que l'approche stochastique résout le problème robuste plus rapidement et indépendamment du budget d'incertitude Γ .



STG_construction_heuristics/images/exp3_evolution_lissee.png

FIGURE 6 – Smoothed progression between KCRDKL & KCHOGL relative to N .

La Figure ?? décompose les temps d'exécution entre le Maître et le Sous-Problème par rapport à N . Alors que nous pensions que le Maître pourrait compenser l'augmentation du temps passé dans le Sous-Problème, ce n'est pas le cas. Tandis que le temps du Sous-Problème de KCRDKL reste parfaitement insensible à l'augmentation du nombre de scénarios N , celui de KCHOGL croît. L'algorithme **MaxMin** impose de calculer la distance entre chaque candidat et chaque chemin déjà sélectionné. Ce coût algorithmique pénalise considérablement l'extraction des chemins optimaux.

FIGURE 7 – Impact of N and L on performance.

La Figure ?? compare l'évolution moyenne du temps de calcul global ainsi que l'évolution du nombre moyen d'itérations global en fonction de deux métriques : le nombre de chemins remontés au Maître et la limite d'exploration du DFS. L'interprétation du nombre moyen d'itérations démontre que KCHOGL permet effectivement, dans certaines configurations très spécifiques, de réduire marginalement le nombre d'itérations du Maître par rapport à un tirage aléatoire. Cependant, cet écart est relativement infime. L'information spatiale apportée par KCHOGL n'est donc pas suffisante pour lui permettre de battre l'approche stochastique.

En conclusion, lorsque nous fixons $N > 2$, l'approche par tirage aléatoire est le meilleur compromis. La topologie de l'espace d'incertitude du problème de *Lot-Sizing* ne justifie pas le calcul de la distance de Manhattan ; un simple tirage aléatoire priorisé s'avère plus robuste et plus performant.

7.4. Expérience 4 : apports de la méthode BAE

L'objectif de cette expérimentation est d'observer les performances de la méthode BAE (ou KC dans la Figure ??), proposée par [1]. Elle utilise la compilation de connaissances ainsi qu'une

heuristique gloutonne pour renvoyer des scénarios⁸. La sélection des chemins à renvoyer au Maître se fait au travers d'une heuristique nommée *SelectAll/SelectFew*. L'idée principale de cette heuristique est la suivante. Chaque chemin l , reconstruit par la *backward*, est ajouté à l'ensemble des scénarios renvoyés au maître $\mathcal{U}$ suivant une loi de Bernoulli de paramètre p fixé. Concrètement, *SelectAll* correspond à $p_{All} = 1$, c'est à dire que nous renvoyons tous les chemins optimaux (et sous-optimaux, cela dépend du paramètre τ). Cette heuristique remonte donc l'intégralité des connaissances, ce qui peut ralentir le problème Maître, mais peut également permettre d'effectuer moins d'itérations. De l'autre côté, nous avons $0 < p_{Few} < 1$, qui correspond à *SelectFew* (dans notre étude, nous avons fixé empiriquement $p_{Few} = 0,35$). Il est tout à fait cohérent que, plus p diminue, moins l'adversaire aura de connaissances à remonter au Maître. Un gain de temps doit être envisageable, accompagné d'une potentielle augmentation du nombre d'itérations.

Pour cette expérience, le paramètre de relaxation a été fixé à $\tau = 1$, et la limite d'exploration du DFS a été fixée à $limit_DFS = 600$. Nous évaluons 7 méthodes. BA et KCU ne renvoient qu'un scénario par itération. KCUD, KCHOGL et KCRDKL en renvoient chacun 2, tandis que KC_{All} et KC_{Few} ne limitent pas le nombre de scénarios renvoyés.

8. Cette heuristique est détaillée plus amplement dans le rapport.

Γ	BA		KA_{All}		KC_{Few}	
	time	iter	time	iter	time	iter
1	0,002	3,16	0,052	1,26	0,049	1,26
21	0,291	8,56	0,058	1,0	0,059	1,0
41	0,393	8,5	0,202	1,0	0,194	1,0
61	0,537	8,44	0,338	1,0	0,332	1,0
81	0,769	8,66	0,537	1,0	0,532	1,0
101	1,031	8,62	0,816	1,0	0,816	1,0

TABLE 1 – Extrait du temps moyen et du nombre moyen d'itérations entre BA, KC_{All} & KC_{Few} pour $\Gamma \in \{1, 21, 41, 61, 81, 101\}$.

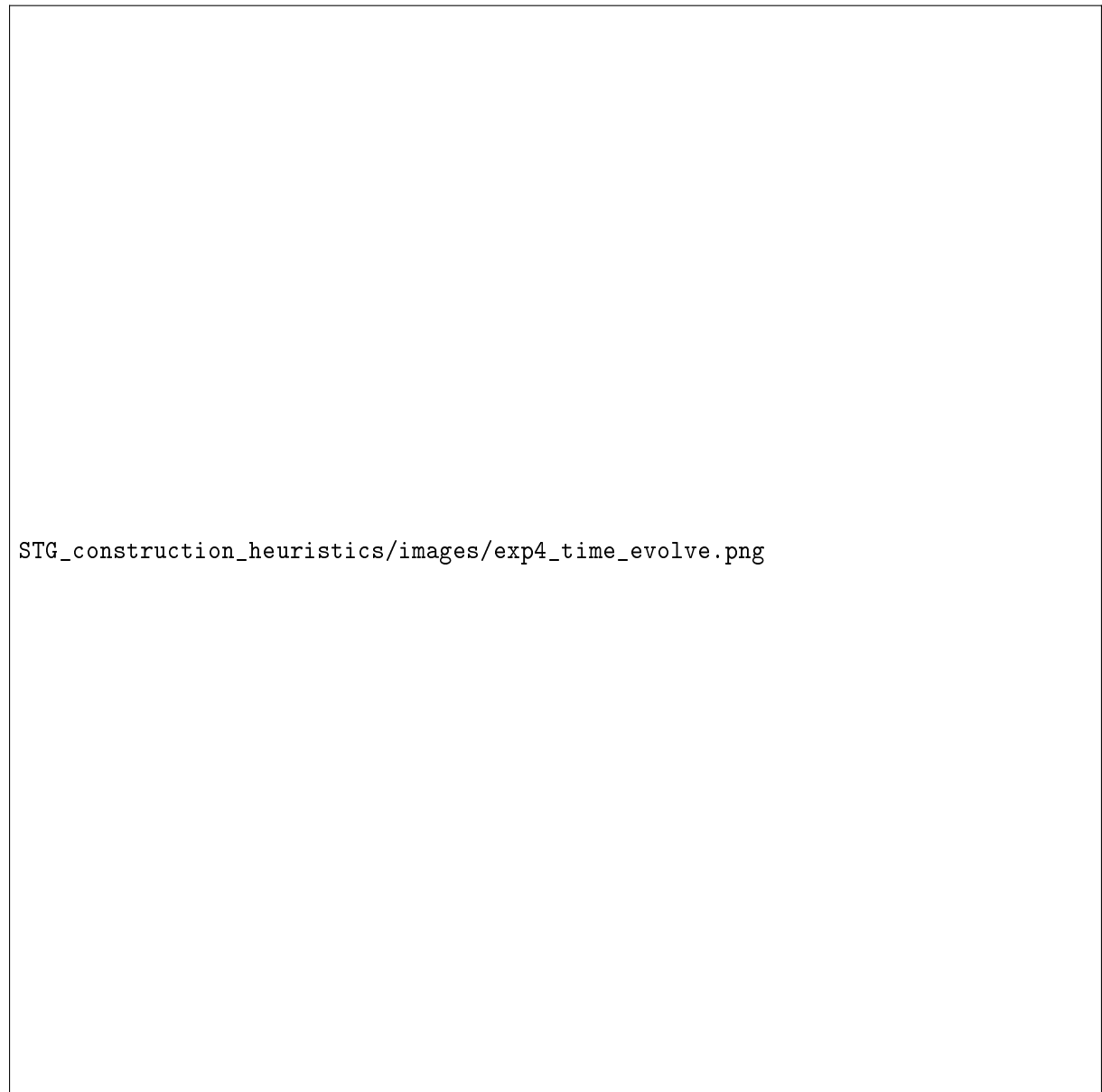


FIGURE 8 – Comparison of sub-problem and master problem times during peak period.

Nous pouvons observer, dans la Figure ??, que la méthode KC_{Few} est la moins performante en termes de temps total moyen. Comme attendu, elle est relativement meilleure que la méthode

Γ	BA		KCHOGL		KC_{Few}		KDRDKL		KCU		KCUD	
	time	iter	time	iter	time	iter	time	iter	time	iter	time	iter
1	0,02	3,16	0,006	1,6	0,005	1,26	0,006	1,64	0,006	1,82	0,006	1,52
21	0,29	8,56	0,03	1,22	0,059	1,0	0,032	1,26	0,03	1,24	0,031	1,22
41	0,393	8,5	0,099	1,3	0,194	1,0	0,098	1,3	0,096	1,32	0,093	1,22
61	0,537	8,44	0,195	1,3	0,332	1,0	0,195	1,3	0,194	1,32	0,184	1,22
81	0,769	8,66	0,335	1,32	0,532	1,0	0,335	1,32	0,329	1,32	0,316	1,24
101	1,031	8,62	0,535	1,34	0,816	1,0	0,533	1,32	1,528	1,34	0,508	1,26

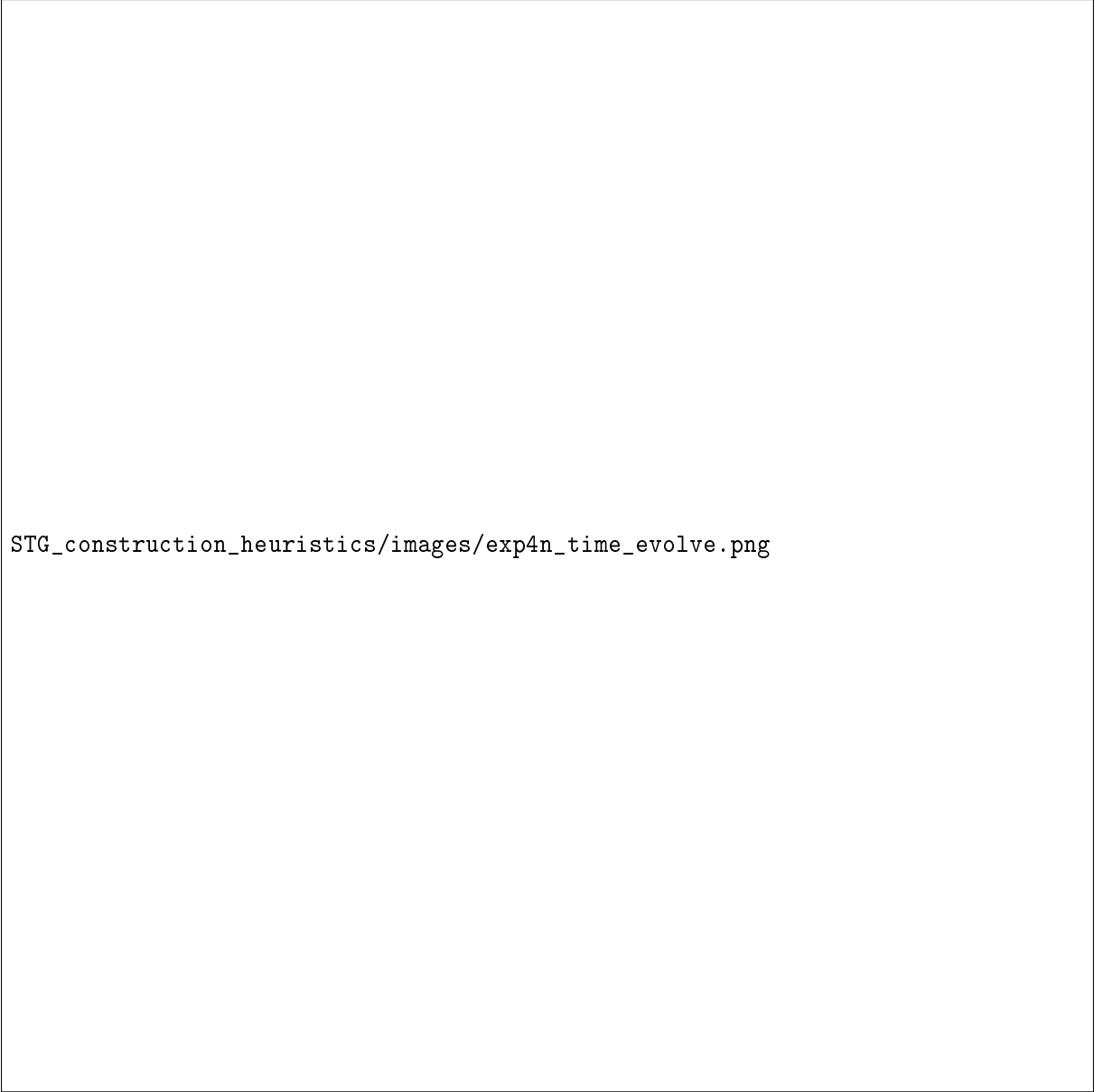
TABLE 2 – Extrait du temps moyen et du nombre moyen d'itérations des 6 méthodes pour $\Gamma \in \{1, 21, 41, 61, 81, 101\}$.

BA sur de petits Γ , mais s'effondre complètement lorsque l'incertitude maximale augmente. Toutefois, nous pouvons noter que son Sous-Problème est le plus performant. En effet, cela s'explique par sa construction ; il ne perd pas de temps dans la sélection complexe des chemins à renvoyer. Cependant, il renvoie un nombre de chemins beaucoup trop important, ce qui alourdit le temps de calcul du Maître. De plus, nous pouvons considérer la méthode KCRDKL comme une amélioration de BAE, car elle conserve la même structure en ne renvoyant qu'un petit nombre N de scénarios par itération. Au-delà du temps d'itération, notre conjecture était correcte sur le fait que KC_{Few} permettait de sauver des itérations (voir Tableau ??). C'est la méthode qui en effectue le moins parmi toutes celles testées. Ce résultat est cohérent puisque l'afflux important de connaissances permet au Maître de converger plus rapidement.

7.5. Expérience 5 : performances au regard de la périodicité

Le but de cette expérience est d'observer comment évoluent les performances des différentes heuristiques au regard de la périodicité. Comme indiqué dans la Section ??, nous avons une probabilité p_{dp} qu'une référence j soit demandée à une période t , en période pleine, qui est égale à $p_{dp} = 0,7$, et à $1 - p_{dp} = 0,3$ en période creuse. Chaque période a une durée de 13 périodes sur les 52 au total.

Les paramètres de la simulation sont les mêmes que pour l'Expérimentation ?. Nous testons les mêmes heuristiques, seules les instances changent.



STG_construction_heuristics/images/exp4n_time_evolve.png

FIGURE 9 – Comparison of sub-problem and master problem times during off-peak period for KCUD, KCHOGL & KCRDKL.

L'absence de périodicité dans la génération des instances influe sur le classement final. Cependant, les heuristiques KCHOGL, KCRDKL, KCU et KCUD semblent insensibles à ce changement. Elles sont toutes les quatre dans le même ordre de grandeur. Nous pouvons noter que BA et KC ont été plus performants que dans l'Expérience ??, dans pour autant concurrencer les autres heuristiques. Cela est cohérent, car l'absence de périodicité rend le problème plus facile.

8. Conclusion

Cette étude a présenté et évalué plusieurs heuristiques de compilation de connaissances pour la résolution robuste de problèmes de *Lot-Sizing* via la décomposition de Benders. L'objectif de cette analyse était d'identifier la stratégie d'extraction des scénarios adverses la plus efficace, en termes de compromis entre la qualité des coupes générées et le coût computationnel associé.

Après avoir réalisé plusieurs expériences, nous pouvons retenir trois conclusions quant à la manière la plus efficace de modéliser et d'extraire l'information adverse. L'intégration des pires scénarios sous forme de chemins au sein d'un graphe partiel de budget permet au problème Maître de limiter l'explosion combinatoire. L'expérience menée sur $N = 2$ scénarios a démontré que forcer l'adversaire aux extrêmes temporels génère des coupes plus discriminantes. Même si l'approche KCHOGL nous semblait intéressante d'un point de vue mathématique, elle s'est révélée sous-optimale, apportant un gain total très marginal. À l'inverse, la baseline stochastique KCRDKL s'impose comme une méthode plus robuste pour des quotas de sélection plus larges ($N > 2$).

La limite de la méthode aléatoire KCRDKL est qu'elle est « aveugle ». En tirant les scénarios de manière uniforme dans l'ensemble des chemins candidats, nous traitons toutes les attaques de la même façon. Or, dans la réalité, certaines configurations d'incertitude ont beaucoup plus de chances de dégrader la valeur objective du Maître que d'autres.

Aussi, nous pouvons y voir une ouverture vers l'Apprentissage par Renforcement (*Reinforcement Learning* - RL). L'idée serait de remplacer ce tirage uniforme par un agent intelligent capable d'apprendre une politique de sélection $\pi(a|s)$. Concrètement, le modèle s'articulerait de la façon suivante. L'état s serait représenté par la topologie du graphe de budget partiel généré par une des méthodes KC. L'action a consisterait à cibler et pondérer les scénarios adverses les plus pertinents parmi les candidats. Enfin, la score final serait mesuré par la capacité, de la coupe de Benders générée, à dégrader fortement la borne inférieure du Maître.

Etant donné que notre graphe de budget est un DAG, l'utilisation d'un Réseau de Neurones Graphiques (GNN) comme approximateur de fonction de valeur serait pertinent. Les GNN sont conçus pour traiter ces structures de données non-euclidiennes et pourraient encoder les contraintes de capacité locales.

Contrairement à KCHOGL, nous nous affranchissons de la complexité en $\mathcal{O}(N^2)$ liée au calcul des distances L_1 . Contrairement à KCRDKL, l'extraction n'est plus aléatoire. L'agent apprendrait à identifier les scénarios critiques, couplant ainsi la vitesse d'extraction de la compilation de connaissances avec l'intelligence du *Machine Learning*.

Références

- [1] T. PORTOLEAU. « Représentations discrètes pour l'ordonnancement et la planification robustes ». Theses. Université toulouse Jean Jaurès, juin 2022. URL : <https://laas.hal.science/tel-04631599>.